



Chapter 13:

Abstract Classes and Interfaces

Part-II

13.6. The Comparable Interface

👉 What is Comparable Interface?

- ✨ The `Comparable` interface lets objects be compared and sorted.
- ✨ Classes implement it to define their own natural order.
- ✨ It has one method: `compareTo`, which compares the current object with another.
- ✨ Used for sorting in Java (e.g., with `Arrays.sort`).

👉 Comparable Interface

✨ The Comparable interface defines the compareTo method for comparing objects.

Syntax:

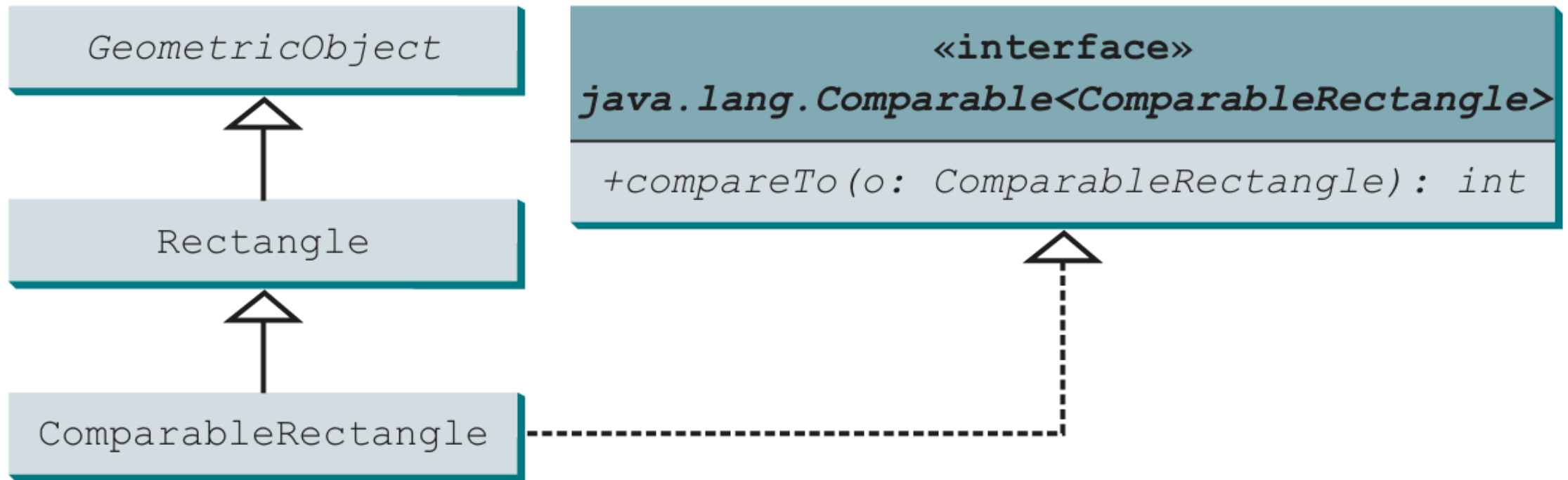
```
package java.lang;  
public interface Comparable<E> {  
    int compareTo(E o);  
}
```

Explain:

✨ `E` is the type of objects that this object may be compared to.

✨ `compareTo(E o)` method compares the current object with the specified object and returns: Negative if less than, zero if equal, positive if greater than the specified object.

Example: UML Diagram for Comparable Interface



Example: Comparable Rectangle Area

```
public interface Comparable<T> {  
    int compareTo(T o);  
    String toString();  
}  
  
public class Rectangle implements Comparable<Rectangle> {  
    private double width, height;  
    public Rectangle(double w, double h) { width = w; height = h; }  
    public double getArea() { return width * height; }  
    @Override  
    public int compareTo(Rectangle o) {  
        return Double.compare(getArea(), o.getArea());  
    }  
    @Override  
    public String toString() {  
        return "Width: " + width + ", Height: " + height + ", Area: " + getArea();  
    }  
}
```

Explain:

- ✨ The `Rectangle` class implements the `Comparable<Rectangle>` interface.
- ✨ The `compareTo` method compares the area of the current rectangle with another rectangle.
- ✨ The `toString` method provides a string representation of the rectangle's dimensions and area.

TIP: Sort Rectangles

Example: Sorting an array of `Rectangle` objects

```
import java.util.Arrays;
public class TestComparable {
    public static void main(String[] args) {
        Rectangle[] rectangles = {
            new Rectangle(3, 4),
            new Rectangle(2, 5),
            new Rectangle(1, 6)
        };
        Arrays.sort(rectangles);
        for (Rectangle r : rectangles) {
            System.out.println(r);
        }
    }
}
```

Output:

```
Width: 1.0, Height: 6.0, Area: 6.0  
Width: 2.0, Height: 5.0, Area: 10.0  
Width: 3.0, Height: 4.0, Area: 12.0
```

Explain:

- ✨ The `Arrays.sort` method sorts the array of `Rectangle` objects based on their area using the `compareTo` method defined in the `Rectangle` class.
- ✨ The output shows the rectangles sorted in ascending order of their area.

13.7. The Cloneable Interface

👉 What is Cloneable Interface?

- ✨ The `Cloneable` interface is a marker in Java that lets objects be cloned.
- ✨ It has no methods—just tells Java the class can be copied.
- ✨ To clone an object, the class must implement `Cloneable` and override the `clone()` method from `Object`.
- ✨ If you try to clone without `Cloneable`, Java throws an error.

👉 Cloneable Interface

✨ The Cloneable interface specifies that an object can be cloned.

Syntax:

```
package java.lang;  
public interface Cloneable {  
    // No methods defined, it's a marker interface  
}
```

Explain:

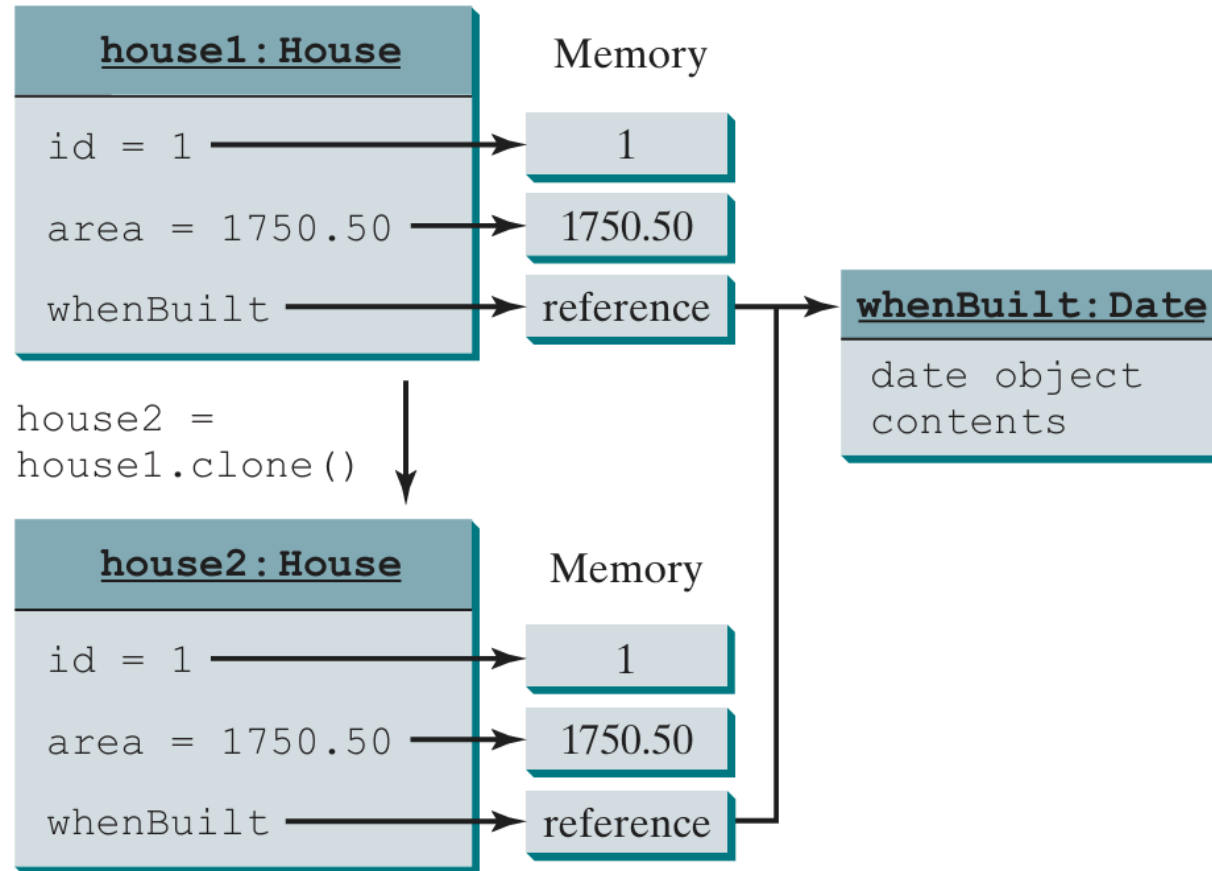
- ✨ The `Cloneable` interface is a marker interface with no methods.
- ✨ A class must implement the `Cloneable` interface to indicate that its instances can be cloned.
- ✨ The `clone()` method is defined in the `Object` class and must be overridden in the subclass to provide the cloning functionality.

Example: Cloneable House Class

```
import java.util.Date;
public class House implements Cloneable {
    private int id;
    private double area;
    private Date whenBuilt;
    public House(int id, double area) {
        this.id = id;
        this.area = area;
        this.whenBuilt = new Date();
    }
    public Object clone() {
        return super.clone();
    }
}
```

👉 Shallow Cloning

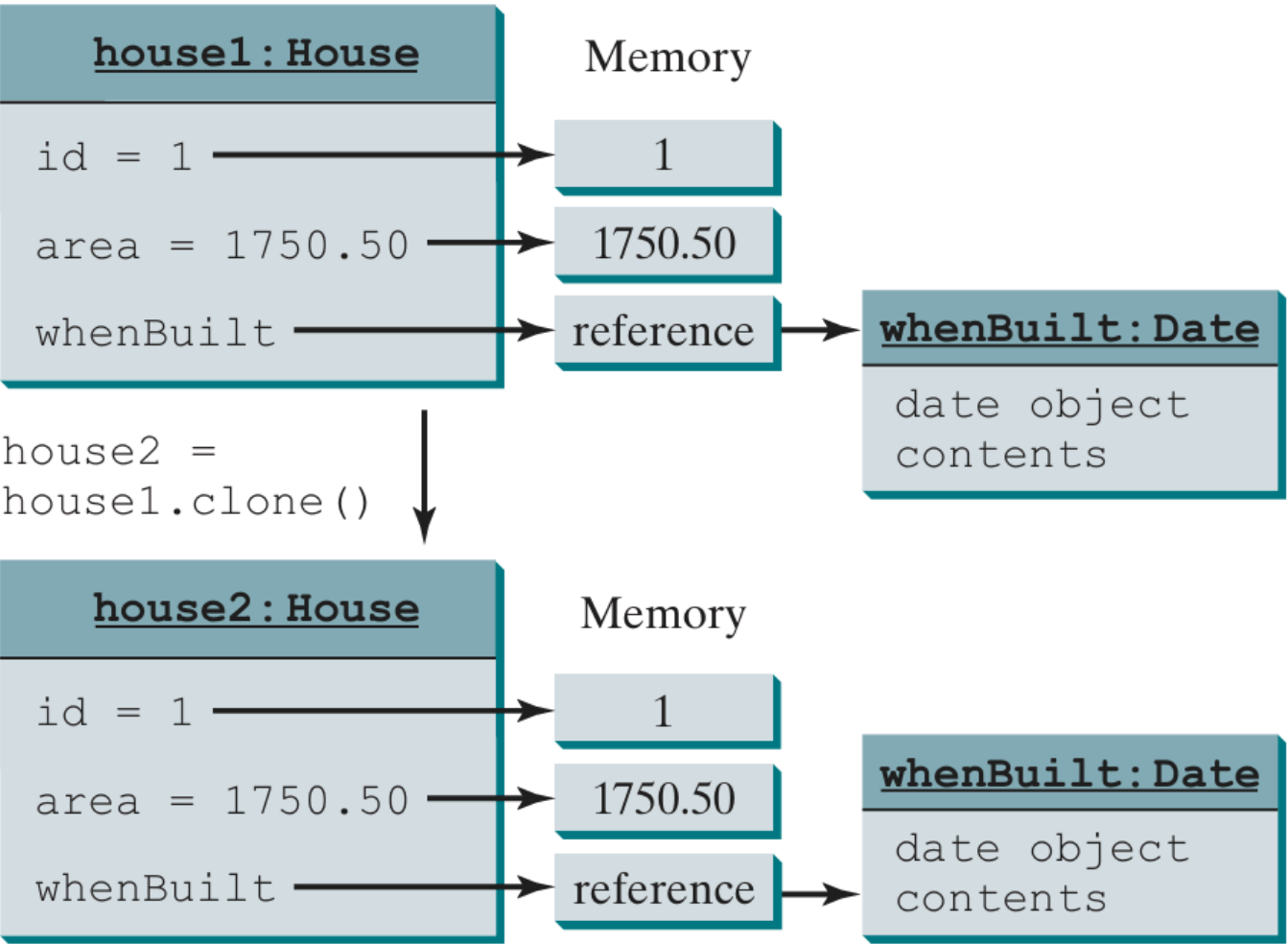
Example: Shallow Cloning of House Class



Snippet:

```
@Override  
protected Object clone() {  
    return super.clone(); // Shallow copy  
}
```

👉 Deep Cloning



Snippet:

```
@Override
protected Object clone() {
    House h = (House) super.clone(); // Shallow copy
    h.whenBuilt = (Date) whenBuilt.clone(); // Deep copy of mutable field
    return h;
}
```

13.8. Usage of Abstract Class vs. Interface

👉 When to Use Abstract Classes vs. Interfaces?

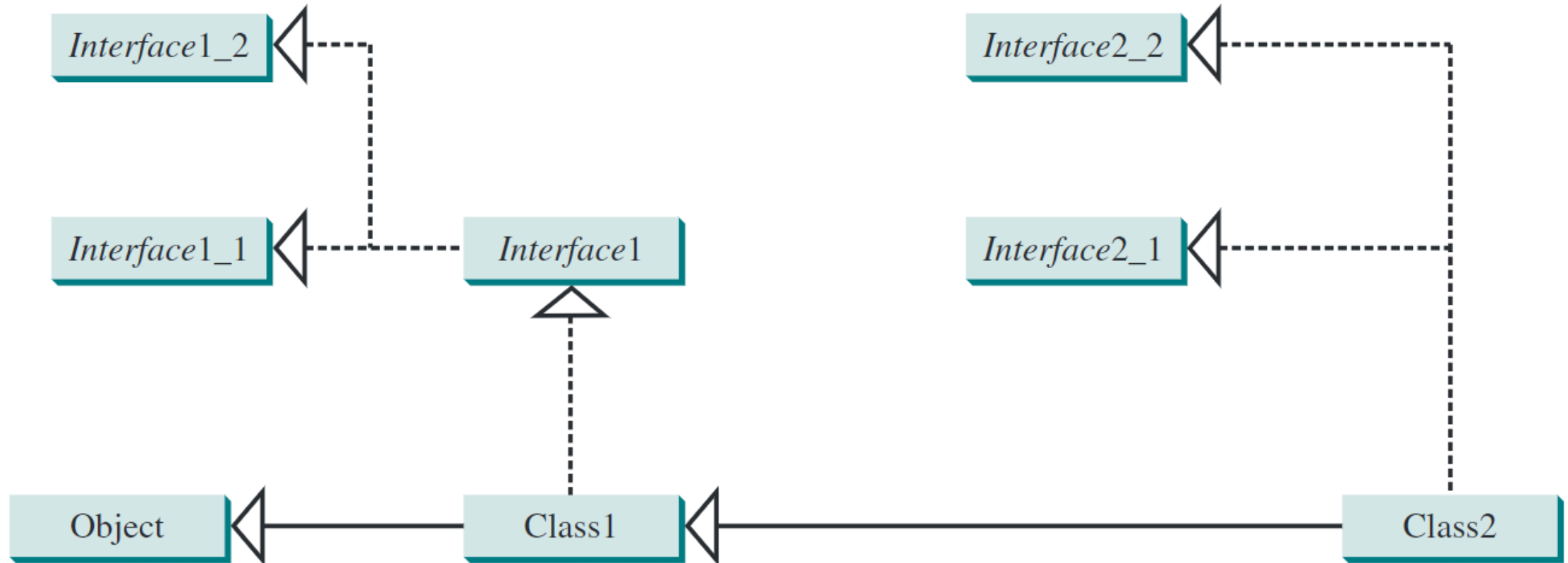
✨ Abstract Classes:

- ✨ Share code and fields among related classes
- ✨ Provide default method implementations
- ✨ Can have constructors and maintain state

✨ Interfaces:

- ✨ Define a contract for any class
- ✨ Allow multiple inheritance of behavior
- ✨ Require implementing classes to define methods
- ✨ Can have default methods and constants

👉 Visualization of Interface



👉 Best Practice Abstract Class with Interface

Example: Combination of Abstract Class and Interface

```
public abstract class Animal { // Abstract class for animals
    protected String name;
    public Animal(String name) { this.name = name; }
    public abstract void makeSound(); // Abstract method for making sound
}

public interface Pet { // Marker interface for pets
    void play(); // Abstract method for pets
}

public class Dog extends Animal implements Pet {
    public Dog(String name) { super(name); }
    @Override
    public void makeSound() { System.out.println(name + " says Woof!"); }
    @Override
    public void play() { System.out.println(name + " is playing fetch!"); }
}
```

Abstract Class vs. Interface Comparison

Feature	Abstract Class	Interface (Java 8+)
Methods	Abstract & concrete	Abstract, default, static
Fields	Instance & static	Static final (constants)
Constructors	Allowed	Not allowed
Multiple Inheritance	Not supported	Supported
Access Modifiers	Any	Public (default)
Use Case	Share code (related classes)	Define contract (unrelated classes)
Inheritance Keyword	<code>extends</code>	<code>implements</code>
Instantiation	Not directly	Not directly

13.9. Case Study: The Rational Class

Practice.

13.10. Class-Design Guidelines

Class-Design Guidelines

✨ When designing classes, consider the following guidelines to ensure good design practices:

✨ **Step 1:** Ensure your class has high cohesion.

✨ **Step 2:** Maintain consistency in naming and structure.

✨ **Step 3:** Apply encapsulation to protect data.

✨ **Step 4:** Strive for clarity in your code.

✨ **Step 5:** Make your class complete for its intended purpose.

✨ **Step 6:** Decide between instance and static members.

✨ **Step 7:** Choose between inheritance and composition.

✨ **Step 8:** Decide whether to use interfaces or abstract classes.

Cohesion

Example: High Cohesion

```
public class Circle {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
  
    public double getCircumference() {  
        return 2 * Math.PI * radius;  
    }  
}
```

Explain:

- ✨ The `Circle` class is focused only on circle-related tasks like area and circumference.
- ✨ All its methods are about circles, making it easy to understand and use.
- ✨ This clear focus makes the code simple and reusable.

Consistency

Example: Consistent Naming

```
public class Rectangle {  
    private double width, height;  
    public Rectangle(double width, double height) {  
        this.width = width; this.height = height;  
    }  
    public double getArea() { return width * height; }  
    public double getPerimeter() { return 2 * (width + height); }  
}
```

Explain:

- ✨ The `Rectangle` class uses clear and consistent names for its fields and methods.
- ✨ Names like `width` , `height` , `getArea()` , and `getPerimeter()` make the code easy to read and understand.
- ✨ Consistent naming helps everyone quickly know what each part of the class does.
- ✨ This makes the code easier to maintain and work on with others.

Encapsulation

Example: Encapsulation

```
public class BankAccount {  
    private String accountNumber;  
    private double balance = 0.0;  
  
    public BankAccount(String accountNumber) {  
        this.accountNumber = accountNumber;  
    }  
    public void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) balance -= amount;  
    }  
    public double getBalance() { return balance; }  
}
```

Explain:

- ✨ The `BankAccount` class uses encapsulation by making its fields `private` and exposing only public methods (`deposit` , `withdraw` , `getBalance`) for controlled access.
- ✨ This protects the internal state, ensures data integrity, and allows validation or logic changes without affecting external code.



Example: Clarity in Code

```
public class TemperatureConverter {  
    public static double celsiusToFahrenheit(double celsius) {  
        return (celsius * 9/5) + 32;  
    }  
  
    public static double fahrenheitToCelsius(double fahrenheit) {  
        return (fahrenheit - 32) * 5/9;  
    }  
}
```

Explain:

- ✨ The `TemperatureConverter` class is clear and easy to understand.
- ✨ Method names like `celsiusToFahrenheit` and `fahrenheitToCelsius` show exactly what each does.
- ✨ The code is simple, with each method doing just one thing.
- ✨ This makes it easy to use, read, and maintain.

Completeness

Example: Complete Class Design

```
public class Student {
    private String name, studentId;
    private int age;

    public Student(String name, int age, String studentId) {
        this.name = name; this.age = age; this.studentId = studentId;
    }
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getStudentId() { return studentId; }
    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age + ", Student ID: " + studentId);
    }
}
```

Explain:

- ✨ The `Student` class has all the basic details and actions for a student.
- ✨ It stores the name, age, and student ID, and has methods to get this info and show it.
- ✨ This makes it easy to use the class for student records or displaying details.
- ✨ The class is simple, clear, and ready to use in real programs.

Instance vs. Static Members

Example: Instance vs. Static Members

```
public class MathUtils {  
    public static final double PI = 3.14159;  
  
    public static double square(double number) {  
        return number * number;  
    }  
  
    public double cube(double number) {  
        return number * number * number;  
    }  
}
```

Explain:

- ✨ `PI` and `square` are static, so you can use them without making a `MathUtils` object.
- ✨ `cube` is an instance method, so you need to create a `MathUtils` object to use it.
- ✨ Use static members for general utilities or constants; use instance members when you need object-specific data.

Inheritance vs. Composition

Example: Inheritance vs. Composition

```
class Animal { void makeSound() { System.out.println("Animal sound"); } }
class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Bark"); }
}
class Person {
    private Dog pet;
    Person(Dog pet) { this.pet = pet; }
    void interactWithPet() { pet.makeSound(); }
}
```

Explain:

- ✨ `Dog` inherits from `Animal` (inheritance: "is-a" relationship).
- ✨ `Person` has a `Dog` as a member (composition: "has-a" relationship).
- ✨ Inheritance is for sharing behavior; composition is for using features of another class.
- ✨ Composition makes code more flexible and reusable.
- ✨ With composition, `Person` can have any pet, not just a `Dog`.

👉 Interfaces vs. Abstract Classes

Example: Interfaces vs. Abstract Classes

```
interface Drawable { void draw(); }
abstract class Shape {
    protected String color;
    Shape(String color) { this.color = color; }
    abstract double area();
}
class Circle extends Shape implements Drawable {
    private double radius;
    Circle(double r, String c) { super(c); radius = r; }
    @Override double area() { return Math.PI * radius * radius; }
    @Override public void draw() {
        System.out.println("Drawing a " + color + " circle, radius " + radius);
    }
}
```

Explain:

- ✨ `Drawable` is an interface with a `draw` method.
- ✨ `Shape` is an abstract class with a color and an abstract `area` method.
- ✨ `Circle` extends `Shape` and implements `Drawable`, so it must define both `area` and `draw`.
- ✨ This shows how a class can use both an abstract class (for shared code) and an interface (for extra abilities).
- ✨ Interfaces add more features; abstract classes share common code.
- ✨ This makes your code more flexible.

End of the Chapter